S10.2 · Redes Densas con Keras

Aprende a construir, compilar y entrenar redes neuronales completamente conectadas usando Keras sobre TensorFlow. En esta sesión cubriremos desde los fundamentos de los tensores hasta el despliegue de modelos con datos tabulares reales.

DEEP LEARNING KERAS TENSORFLOW



¿Qué es TensorFlow y qué es Keras?

TensorFlow

Biblioteca de cómputo numérico de código abierto desarrollada por Google. Permite ejecutar operaciones matemáticas sobre tensores de forma eficiente, tanto en CPU como en GPU y TPU. Es el **motor de bajo nivel** que impulsa los cálculos.

Keras

API de alto nivel integrada en TensorFlow (`tf.keras`) que simplifica enormemente la construcción de redes neuronales. Proporciona capas, optimizadores y utilidades listas para usar, siguiendo el principio de **legibilidad y rapidez de prototipado**.

Desde TensorFlow 2.0, Keras es la interfaz oficial recomendada. No es necesario usar la API de bajo nivel para la mayoría de proyectos.

Atributos clave de un tensor

Propiedades esenciales

shape

Las dimensiones del tensor, por ejemplo `(1000, 28, 28, 1)` para 1000 imágenes de 28×28 píxeles en escala de grises.

dtype

El tipo de dato: `float32` es el más habitual en redes neuronales; `int32` se usa para etiquetas enteras.

rank

El número de ejes o dimensiones. Equivalente a `len(tensor.shape)`.

Operaciones habituales

```
x = tf.ones((3, 4))
y = tf.zeros((3, 4))

# Suma y producto
z = x + y
z = tf.matmul(x, tf.transpose(y))

# Redimensionar
r = tf.reshape(x, (12,))

# Convertir a NumPy
arr = x.numpy()
```

Modelo Sequential

El modelo `Sequential` es la forma más sencilla de construir una red en Keras. Las capas se apilan una tras otra, formando una **pila lineal**. Es perfecto para la mayoría de redes densas clásicas.

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

Dense(64, activation='relu')

Capa oculta con 64 neuronas y activación ReLU.

Dense(32, activation='relu')

Segunda capa oculta, más pequeña, para comprimir la representación.

Dense(1, activation='sigmoid')

Capa de salida para clasificación binaria.

API Funcional

La API funcional permite crear arquitecturas **más complejas**: múltiples entradas, múltiples salidas, conexiones residuales o ramas paralelas. Cada capa es una función que recibe y devuelve tensores.

```
inputs = keras.Input(shape=(10,))
x = layers.Dense(64, activation='relu')(inputs)
x = layers.Dense(32, activation='relu')(x)
outputs = layers.Dense(1, activation='sigmoid')(x)

model = keras.Model(inputs=inputs,
                    outputs=outputs)
```

¿Cuándo usar cada uno?

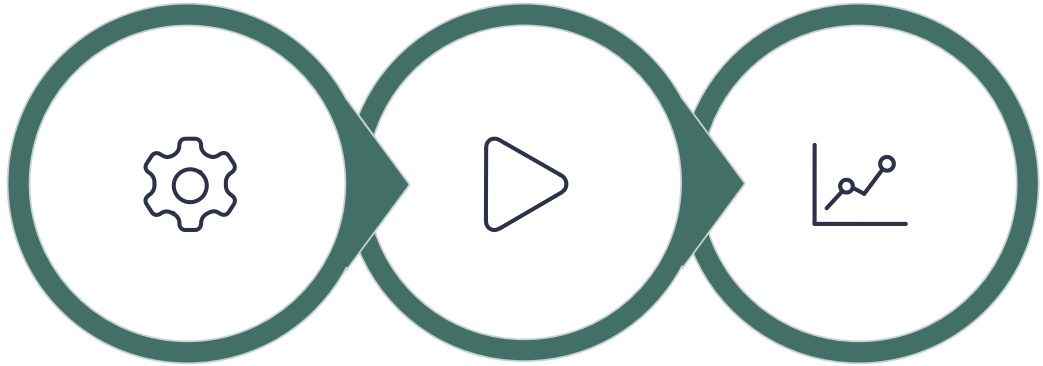
→ **Sequential**

Red simple, lineal, sin bifurcaciones. Ideal para aprender y para la mayoría de casos de uso.

→ **API Funcional**

Redes con múltiples entradas/salidas, skip connections o bloques compartidos.

El ciclo de entrenamiento en Keras



Compilar Entrenar Evaluar

compile()

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

fit()

```
history = model.fit(
 X_train, y_train,
 epochs=20,
 batch_size=32,
 validation_split=0.2
)
```

Resumen: salidas y pérdidas según la tarea

Tarea	Activación de salida	Función de pérdida	Ejemplo
Clasificación binaria	`sigmoid`	`binary_crossentropy`	Spam / no spam
Clasificación multiclase	`softmax`	`categorical_crossentropy`	Clasificar dígitos MNIST
Regresión (una salida)	Ninguna (lineal)	`mse` o `mae`	Precio de una vivienda
Regresión multisalida	Ninguna (lineal)	`mse`	Coordenadas de un objeto

Elegir la pérdida incorrecta es uno de los errores más comunes. Verifica siempre que coincida con el tipo de etiquetas y la activación de salida.

Entendiendo Batches y Epochs

Lote (Batch)

Subconjunto de datos con el que se calcula un único paso de gradiente. Un `batch_size=32` significa que el modelo actualiza sus pesos cada 32 ejemplos. Lotes más pequeños generalizan mejor; lotes más grandes son más estables y veloces.

Época (Epoch)

Un recorrido completo por **todo** el conjunto de entrenamiento. Si tienes 1.000 muestras y `batch_size=32`, una época son aproximadamente 32 pasos de actualización. Más épocas permiten aprender más, pero aumentan el riesgo de sobreajuste (*overfitting*).

32

batch_size típico

Valores habituales: 16, 32, 64, 128.

10–100

Epochs habituales

Depende del dataset y la tarea.

1

Paso de gradiente

Por cada lote procesado.

CAPÍTULO 7

Guardar y Cargar Modelos

Un modelo entrenado representa horas de cómputo. Keras proporciona métodos sencillos para **persistir** y **recuperar** modelos completos o solo sus pesos.

Guardar el modelo completo

```
model.save('mi_modelo.keras')
```

Guarda arquitectura, pesos y configuración del optimizador en un único fichero. **Recomendado** para la mayoría de casos.

Cargar el modelo

```
model = keras.models.load_model('mi_modelo.keras')
```

Restaura el modelo exactamente como estaba, listo para seguir entrenando o hacer predicciones.

Guardar solo los pesos

```
model.save_weights('pesos.weights.h5')
model.load_weights('pesos.weights.h5')
```

Útil cuando ya tienes la arquitectura definida en código y solo quieres guardar los valores aprendidos.

S10 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S10_02_mlp_keras — 60 min

MLP en Keras: clasificación y regresión

sprints/S10/notebooks/S10_02_mlp_keras.ipynb